

▼ home

> __pycache__

> fixtures

> migrations

> templates

🔗 __init__.py

🔗 admin.py

🔗 apps.py

🔗 forms.py

🔗 models.py

🔗 test_forms.py

🔗 test_views.py

🔗 urls.py

🔗 views.py



CI Python Linter

```
1 from django.contrib import admin
2 from django_summernote.admin import SummernoteModelAdmin
3 from .models import CoachingPost, ProgressComment
4
5
6 @admin.register(CoachingPost)
7 class CoachingPostAdmin(SummernoteModelAdmin):
8     """
9     Lists fields for display in admin, fields for search,
10     field filters, fields to prepopulate, and fields with rich-text editor.
11     """
12     list_display = ('title', 'author', 'status', 'created_on', 'updated_on',)
13     search_fields = ['title', 'excerpt', 'content']
14     list_filter = ('author', 'status', 'created_on', 'updated_on',)
15     prepopulated_fields = {'slug': ('title',)}
16     summernote_fields = ('content',)
17
18
19 # Register your models here.
20 @admin.register(ProgressComment)
21 class ProgressCommentAdmin(admin.ModelAdmin):
22     """
23     Lists fields for display in admin, fields for search,
24     and field filters.
25     """
26     list_display = ('body', 'post', 'author', 'created_on', 'updated_on',)
27     search_fields = ['body', ]
28     list_filter = ('post', 'author', 'created_on', 'updated_on',)
29
```

Settings:



Results:

All clear, no errors found



CI Python Linters

```
1 from django.apps import AppConfig
2
3
4 class HomeConfig(AppConfig):
5     default_auto_field = 'django.db.models.BigAutoField'
6     name = 'home'
7
```

Settings:



Results:

All clear, no errors found



CI Python Lint

```
1 from .models import ProgressComment
2 from django import forms
3
4
5 class CommentForm(forms.ModelForm):
6     """
7     Form class for client users to submit progress comments on a coaching post
8     """
9     class Meta:
10         model = ProgressComment
11         fields = ('body',)
12
```

Settings:



Results:

All clear, no errors found



CI Python Linter

```
29         return f"{a}{b}"
30
31
32 class ProgressComment(models.Model):
33     """
34     Stores a single progress comment entry related to :model:`auth.User`
35     and :model:`home.CoachingPost`.
36     """
37     post = models.ForeignKey(
38         CoachingPost,
39         on_delete=models.CASCADE,
40         related_name="comments",
41     )
42     author = models.ForeignKey(
43         User,
44         on_delete=models.CASCADE,
45         related_name="commenter",
46     )
47     body = models.TextField()
48     created_on = models.DateTimeField(auto_now_add=True)
49     updated_on = models.DateTimeField(auto_now=True)
50
51     class Meta:
52         ordering = ["-updated_on"]
53
54     def __str__(self):
55         a = "Progress comment by "
56         b = f"{self.author}".title()
57         c = f" | on coaching post {self.post.title}"
58         return f"{a}{b}{c}"
59
```

Settings:



Results:

All clear, no errors found



CI Python Linter

```
1 from django.test import TestCase
2 from .forms import CommentForm
3
4
5 class TestCommentForm(TestCase):
6
7     def test_form_is_valid(self):
8         """Test for the 'body' field"""
9         comment_form = CommentForm({'body': 'This is a great post'})
10        self.assertTrue(comment_form.is_valid(), msg='Form is not valid')
11
12    def test_form_is_invalid(self):
13        """Test for the empty 'body' field"""
14        comment_form = CommentForm({'body': ''})
15        self.assertFalse(comment_form.is_valid(), msg="Form is valid")
16
```

Settings:



Results:

All clear, no errors found



CI Python Linter

```
24         )
25         self.post.save()
26
27     def test_render_coaching_post_detail_page_with_comment_form(self):
28         """Test the coaching_post_detail view."""
29         self.client.force_login(self.user)
30         response = self.client.get(reverse(
31             'coaching_post_detail', args=['coaching-post-title']))
32         self.assertEqual(response.status_code, 200)
33         self.assertIn(b"Coaching post title", response.content)
34         self.assertIn(b"Coaching post content", response.content)
35         self.assertIsInstance(
36             response.context['comment_form'], CommentForm)
37         self.client.logout()
38
39     def test_successful_progress_comment_submission(self):
40         """Test for submitting a progress comment on a coaching post"""
41         self.client.login(
42             username="myUsername", password="myPassword")
43         post_data = {
44             'body': 'This is a test progress comment.'
45         }
46         response = self.client.post(reverse(
47             'coaching_post_detail', args=['coaching-post-title']), post_data)
48         self.assertEqual(response.status_code, 200)
49         self.assertIn(
50             b'Your progress comment has been added successfully',
51             response.content
52         )
53         self.client.logout()
```

Settings:



Results:

All clear, no errors found



CI Python Linters

```
1 from django.urls import path
2 from . import views
3
4
5 urlpatterns = [
6     path('', views.CoachingPostList.as_view(), name='home'),
7     path('<slug:slug>/',
8         views.coaching_post_detail, name='coaching_post_detail'),
9     path('<slug:slug>/edit_comment/<int:comment_id>',
10         views.progress_comment_edit, name='progress_comment_edit'),
11     path('<slug:slug>/delete_comment/<int:comment_id>',
12         views.progress_comment_delete, name='progress_comment_delete'),
13 ]
14
```

Settings:



Results:

All clear, no errors found



CI Python Linter

```
117
118     return HttpResponseRedirect(reverse('coaching_post_detail', args=[slug]))
119
120
121 def progress_comment_delete(request, slug, comment_id):
122     """
123     Delete an individual progress comment.
124
125     **Context**
126
127     ``post``
128         An instance of :model:`home.CoachingPost`.
129     ``comment``
130         A single progress comment related to the post.
131     """
132     comment = get_object_or_404(ProgressComment, pk=comment_id)
133
134     if comment.author == request.user:
135         comment.delete()
136         messages.add_message(
137             request, messages.SUCCESS,
138             'Progress comment deleted!'
139         )
140     else:
141         messages.add_message(
142             request, messages.ERROR,
143             'You can only delete your own progress comments!'
144         )
145
146     return HttpResponseRedirect(reverse('coaching_post_detail', args=[slug]))
147
```

Settings:



Results:

All clear, no errors found